Attention Is All You Need (Transformer)

ref	https://arxiv.org/pdf/1706.03762
type	예습과제
week	week2

예습_순환신경망, Transformer

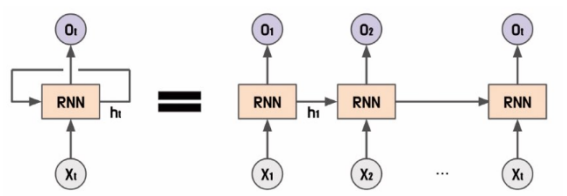
1. 순환 신경망(RNN)

RNN이란

*자연어 처리에서는 문장과 단어의 문맥과 상호작용의 모델링이 정확한 의미 파악이 중요한 요소이다.

- RNN : 연속적인 데이터를 처리하기 위해 개발된 인공 신경망의 한 종류 → 이전에 처리한 데이터를 다음 단계에 활용 & 현재 입력 데이터와 함께 모델 내부에서 과거의 상태를 기억해 현재 상태 예측
- 가변적이고 순서에 의미가 있는 데이터에 활용

RNN의 구조



- 데이터를 순서대로 입력받아 처리 시점마다 은닉 상태로 저장한다.
- 이전 시점의 은닉 상태 h(t-1)와 입력값을 받아 현재 시점 은닉 상태 ht를 계산한다. → 은닉 상태가 다음 시점으로 전달되면서 "기억"의 역할을 한다.

수식 6.11 순환 신경망의 은닉 상태

$$h_t = \sigma_h(h_{t-1}, x_t)$$
$$h_t = \sigma_h(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

- σh는 순환 신경망의 은닉 상태를 계산하기 위한 활성화 함수
- σh는 가중치 W와 편향 b를 이용해 계산한다
- Whh : 이전 시점의 은닉 상태 h(t-1)에 대한 가중치
- Wxh : 입력값 xt에 대한 가중치 / bh : 은닉 상태 ht의 편향

수식 6.12 순환 신경망의 출력값

$$y_t = \sigma_y(h_t)$$
$$y_t = \sigma_y(W_{hy}h_t + b_y)$$

- 각 시점 t에서 현재 입력값 xt와 이전 시점 t-1의 은닉 상태 h(t-1)을 이용해 현재 시점의 은닉 상태 ht와 출력값 yt 를 계산

일대다/다대일/다대다 구조

	일대다 구조	다대일 구조	다대다 구조
구조	하나의 입력 → 여러개의 출력값	여러 개의 입력 → 하나의 출력값	여러 개의 입력 → 여러개의 출력값
특징	출력 시퀀스의 길이를 알고 있어야함		입출력 시퀀스 길이 맞추는 전처리 작업이 필요하다(패딩 추가, 자르기 등) 시퀀스-시퀀스 구조 : 입력 시퀀스를 처리하는 인코더, 출력 시퀀스를 생성하는 디코더
활용	이미지 캡셔닝, 자연어 처리(문장 입력, 각 단어의 품사 예측)	감성 분류, 자연어 추론(문장 분류, 두 문장 간의 관계 추론)	번역기, 음성 인식기

양방향 순환 신경망

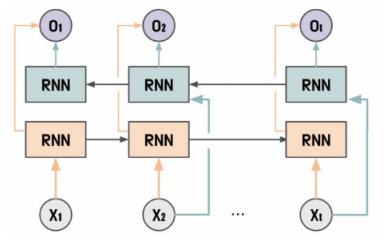


그림 6.16 양방향 순환 신경망 구조

- RNN에서 시간 방향을 양방향으로 처리할 수 있도록 → 이전/이후 시점 데이터 모두 상관관계를 가지고 있다.
- t-1의 은닉 상태뿐만 아니라 t+1의 은닉 상태도 함께 이용
- ex) 문장 사이의 단어 예측하기

다중 순환 신경망

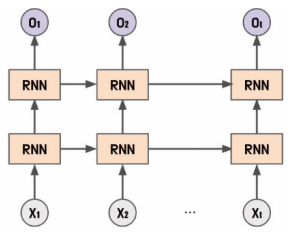


그림 6.17 다중 순환 신경망 구조

- 여러 개의 RNN층을 연결하여 구성한 모델 → 각 RNN층이 서로 다른 정보를 처리하도록 설계
- 데이터의 다양한 특징 추출 가능 → 깊어질수록 복잡한 패턴 학습 가능
- but 학습 시간 오래 걸리고 기울기 소실 문제 가능성도 높아진다.

장단기 메모리 LSTM

> !!RNN의 문제점

장기 의존성 문제 : 특정 시점에서 이전 입력 정보를 이용해 출력값 예측 → 앞선 정보를 끊임없이 반영하기에 데이터의 크기가 커질수록 정보가 충분히 전달되지 X

기울기 소실 또는 기울기 폭주 : 활성화 함수로 인해 역전파 과정에서 발생

[LSTM의 구조]

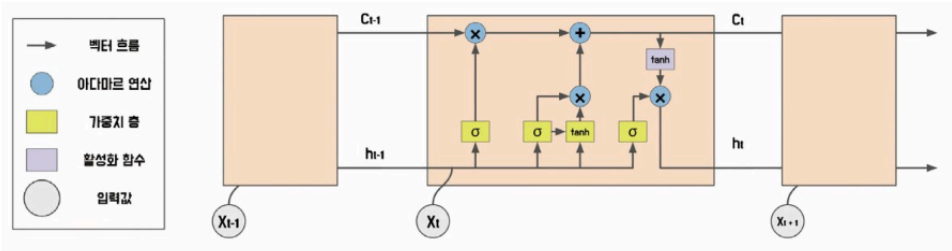


그림 6.18 장단기 메모리 구조

수식 6.15 메모리 셀

$$c_t = f_t \odot c_{t-1} + g_i \odot i_t$$

- Cell state : 메모리 역할 → 정보 저장하고 유지
  - 최종 메모리 셀 계산 방법
  - ⊙ 아다마르 곱 : 원소별 곱셈 연산

수식 6.13 망각 게이트

$$f_t = \sigma(W_x^{(f)} x_t + W_h^{(f)} h_{t-1} + b^{(f)})$$

- Forget gate : 현재 입력과 이전 셀 상태를 입력받아 어떤 정보를 삭제할지 결정
  - 출력값 ft는 시그모이드 활성화 함수를 사용해 계산된다.
  - Wxf와 Whf는 입력값과 은닉 상태를 위한 가중치
  - bf : 망각 게이트의 편향

- 현재 시점 입력값 xt와 이전 시점 은닉 상태 h(t-1)를 이용해 출력값 계산 → 0과 1 사이(0에 가까우면 더 많은 정보 삭제)

수식 6.14 기억 게이트

$$g_i = tanh(W_x^{(g)}x_t + W_x^{(g)}h_{t-1} + b^{(g)})$$
$$i_i = sigmoid(W_x^{(i)}x_t + W_x^{(i)}h_{t-1} + b^{(i)})$$

- Input gate : 현재 입력과 이전 셀 상태를 입력받아 어떤 정보를 추가할지 결정
  - gi : [-1,1] 범위의 값으로 존재 / 이전 시점의 은닉 상태와 현재 시점의 입력값 / 활성화 함수로 하이퍼볼릭 탄젠트 사용
  - it : [0,1]의 값을 가지며 얼마나 많은 정보를 기억할건지 가중치 역할 / 새로운 은닉 상태의 기억 제어/ 활성화 함수로 시그모이드 사용

수식 6.16 출력 게이트

$$o_t = \sigma(W_x^{(o)}x_t + W_h^{(o)}h_{t-1} + b^{(o)})$$

- Output gate : 어떤 부분을 출력할지 결정, 현재 입력과 이전 셀 상태, 새로 추가된 정보
  - 활성화 함수로 시그모이드 활용

2. Transformer

Transformer 모델과 어텐션

- Transformer : 딥러닝 모델 중 하나
  - 자연어 처리 분야에서 많은 성과
  - RNN과 CNN과 달리 어텐션 메커니즘만을 이용해 시퀀스 임베딩 표현
- Transformer의 어텐션 메커니즘
  - 인코더(양방향)와 디코더(단반향) 간의 상호작용으로 입력 시퀀스의 중요한 부분에 초점을 맞추어 문맥을 이해하고 출력을 생성
  - 인코더 : 입력 시퀀스 임베딩 → 고차원 벡터로 변환
  - 디코더 : 인코더의 출력을 입력으로 받아 출력 시퀀스 생성
  - 어텐션 메커니즘은 인코더와 디코더 단어 사이의 상관관계를 계산하여 중요한 정보에 집중

    ⇒ 입력 시퀀스의 각 단어가 출력 시퀀스의 어떤 단어와 관련이 있는지 파악

    ⇒ 번역, 요약문 생성 등의 작업

  → RNN보다 학습 속도 빠르고, 병렬 처리가 가능

  → 문장 전체의 정보 고려하여 길이가 길어져도 성능 유지됨

[Transformer 모델 구조]

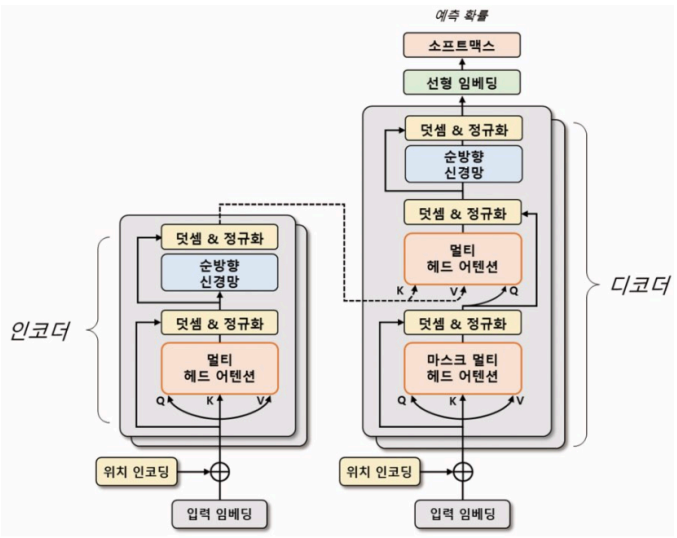


그림 7.2 트랜스포머 모델 구조(Q: 쿼리, K: 키, V: 값)

- 인코더와 디코더는 두 부분으로 구성 → 각각 N개의 Transformer block(Multi-Head Attention+순방향 신경망)으로 구성
- Multi-Head Attention : Q, K, V 벡터를 정의해 입력 시퀀스들의 관계를 Self-Attention하는 벡터 표현 방식
  - Self-Attention? Q, K, V 시작 값이 동일하다는 것(중간의 W에 의해 최종 값은 학습을 통해 달라진다.)
  - **Q와 각 K의 유사도** 계산 → 가중치로 사용하여 값 벡터 합산
  - 계산된 어텐션 행렬은 입력 시퀀스 각 단어의 임베딩 벡터를 대체 → 입력 시퀀스의 단어 사이의 상호작용을 고려해 임베딩 벡터 갱신

- 순방향 신경망
  - Multi-Head Attention에서 산출된 임베딩 벡터를 더욱 고도화하기 위해 사용
  - Multi-Layered, f(Wx+b) 적용
  - 이때의 학습으로 W들은 입력 시퀀스의 각 단어의 의미를 잘 파악할 수 있는 방식으로 개선

- 입력 시퀀스의 처리 : **소스와 타깃**

  ex) 영→한 번역 : 소스(영), 타깃(한)

- 인코더 : 소스 시퀀스 데이터를 위치 인코딩된 입력 임베딩으로 표현 → Transformer 블록의 출력 벡터 생성(입력 시퀀스 데이터의 관계 잘 표현)
- 디코더 : Transformer 블록으로 구성 and **Masked Multi-Head Attention** 활용 → 타깃 시퀀스 데이터를 순차적으로 생성(인코더의 출력 벡터 정보 참조)

입력 임베딩과 위치 인코딩

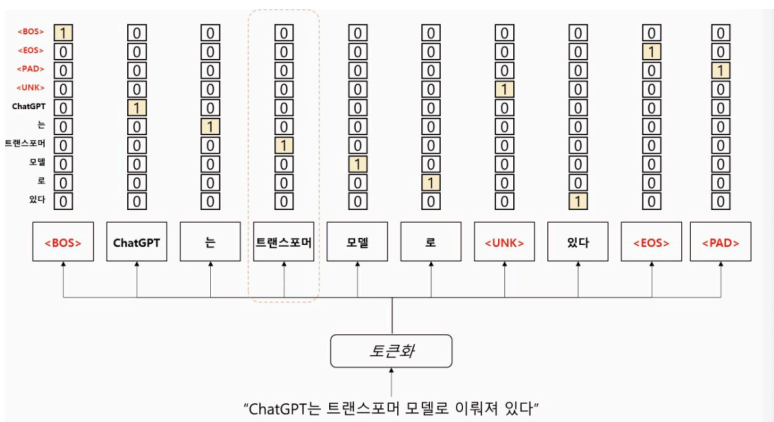
❓위치 인코딩의 필요성 : 입력 시퀀스의 각 단어 → 임베딩 처리 → 벡터 형태 && 입력 시퀀스를 병렬로 처리하여 단어의 순서 정보 제공X
⇒ 위치 정보를 추가해 임베딩 벡터에 순서 정보 모델에 반영

- 위치 인코딩 : 입력 시퀀스의 순서 정보를 모델에 전달하는 방법 → sin, cos 함수로 생성(각 토큰의 위치를 각도로 표현) → 각 토큰 위치마다 임베딩 벡터가 동일하지 X

특수 토큰

Transformer : 단어 토큰 이외의 특수 토큰을 활용하여 문장 표현

→ 입력 시퀀스의 시작과 끝 나타내기 & **Masking area**로 사용(일부 입력 무시 가능)



- BOS : 문장의 시작
- EOS : 문장의 종료
- UNK : 어휘 사전에 없는 단어 → 걍 모르는 단어;;
- PAD : pad임

입력 임베딩으로 변환되는 과정은 Word2Vec 방법과 동일하다. 어휘 사전의 크기를 V, 입력 임베딩 차원을 d라고 했을 때, '트랜스포머'의 원-핫 벡터는 [1, V] 크기를 갖는다. 이 원-핫 벡터는 임베딩 행렬 [V, d]에 의해 [1, d] 크기의 벡터로 변환된다.

이러한 방식을 일반화하면, N개의 문장이 최대 S개의 토큰 길이를 가질 때 [N, S, V] 크기의 원-핫 벡터 텐서는 [N, S, d] 크기의 임베딩 텐서로 변환된다.

트랜스포머 인코더와 디코더

[인코더] : 입력 시퀀스를 임베딩하여 고차원 벡터로 변환(정보 추출 후 다음 계층으로 전달)

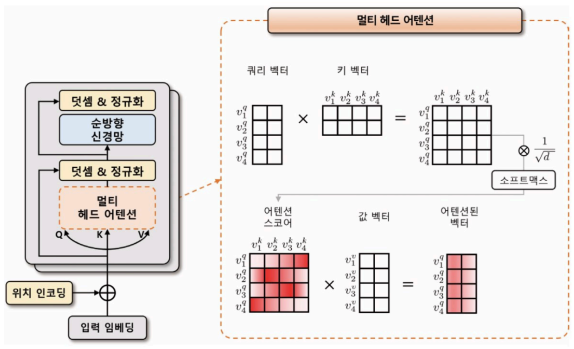


그림 7.5 트랜스포머 인코더 블록 연산 과정

1. 트랜스포머 인코더는 위치 인코딩이 적용된 소스 데이터의 **입력 임베딩** 받기
2. Multi-Head Attention : 입력 텐서 차원이 [N, S, d]라고 할 때 →입력 임베딩은 선형 변환을 통해 3개의 임베딩 벡터 Q, V, K 벡터를 각각 생성
   - Q: 현재 시점에서 참조하고자하는 정보의 위치를 나타내는 벡터
   - K: 쿼리 벡터와 비교되는 대상. 쿼리 벡터를 제외한 입력 시퀀스에서 탐색되는 벡터
   - V: 쿼리 벡터와 키 벡터로 생성된 어텐션 스코어를 얼마나 반영할지 설정하는 가중치 역할

→ Self-Attention : Q, K, V 시작 값이 동일하지만 중간의 W에 의해 최종 값은 학습을 통해 달라진다.

3. Self-Attention : Q, K, 벡터를 내적하고 루트 d만큼 나눠 보정해 어텐션 스코어를 구한다.
4. 어텐션 스코어를 소프트맥스 함수를 이용해 확률적으로 재표현, 이를 V 벡터와 내적해 Self-Attention된 벡터를 생성한다.
5. Multi-Head : 이런 과정을 반복해 셀프 어텐션 스코어맵 생성
   - 입력받는 [N, S, d] 텐서에 k 개의 Self-Attention 벡터를 생성 → Head 에 대한 차원축을 생성해 [N, k, S, d/k] 텐서 형태를 구축→ 생성된 k 개의 Self-Attention 벡터는 임베딩 차원 축으로 다시 병합되어 [N, s, d] 형태로 출력
6. 덧셈 & 정규화 : **Multi-Head Attention**을 통과하기 이전의 입력값 [N, S, d] 텐서와 통과한 이후의 출력값 [N, S, d]텐서를 더함으로써 학습시 발생하는 기울기 소실을 완화 → 이 값에 임베딩 차원 축으로 정규화하는 계층 정규화를 적용
7. 순방향 신경망

- 선형 임베딩 + ReLU or 1차원 합성곱 적용

8. 덧셈 & 정규화 과정이 수행
9. 인코더 블록에서 출력된 벡터는 디코더에 적용 → **Multi-Head Attention**에서 참조되는 K, V 벡터로 활용

[디코더] : 인코더의 출력(위치 인코딩이 적용된 타깃 데이터의 입력 임베딩)을 입력으로 받아 출력 시퀀스 생성 → 입력 시퀀스의 순서 정보 학습 가능

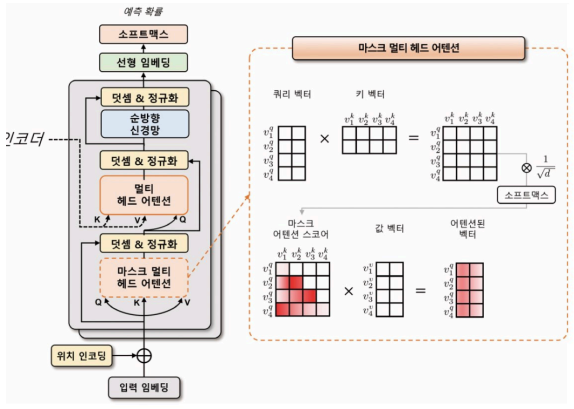


그림 7.6 트랜스포머 디코더 블록 연산 과정

Masked Multi-Head Attention
- 지금까지 생성된 단어들 간 관계 파악
- **인과성(Casuality)**를 반영한 마스크 멀티 헤드 어텐션 모듈
- 현재 위치 이전의 단어들만 참조하게 하여 인과성을 보장함

인코더-디코더 멀티 헤드 어텐션
- 목적: 인코더의 출력을 참고하여 입력 시퀀스와 직접 연결될 수 있게
- K, V 벡터를 인코더의 최종 출력 벡터로부터 사용

논문 리뷰

▼ Terminology

the State of the Art(SOTA)

: 특정분야(NLP, Image Recognition)에 대해 가장 높은 정확도와 성능을 자랑하는 최신 algo

trasduction model

: training&test datasets에서 규칙 추론 → predictive model 생성 X, high cost

induction model

: ML supervised learning 방식 / training datasets에서 규칙 추론 → test set에 적용 → predictive model 생성 , low cost

masking

: decoder stack의 self-attention sub-layer가 이전의 position이 다음의 position에 개입하지 못하도록 변형한 것 → output embedding이 현재 position이 이전 position에만 의존하여 예측하도록(다음꺼는 masking되니까)

*Abstract 참고

왜 진행되었는가? 기존의 encoder•decoder구조+attention의 RNN와 CNN의 한계

어떤 것이 목표인가? RNN•CNN 구조에 attention만 적용한 Transformer라는 새로운 간단한 네트워크 구조 제안

실험에 의한 결론? 영어 구 구문분석에서 크고 제한적인 training data에서도 일반화 가능함.

1. Introduction

💡 논문에서 다루고 있는 주제가 무엇인지와 해당 주제의 필요성이 무엇인가
논문에서 제안하는 방법이 기존 방법의 문제점에 대응되도록 제안 되었는가

논문이 다루는 분야

self-attention 기반의 Transformer

해당 task에서 기존 연구 한계점

‼해결하고자 하는 문제 : 모델 연산에서 sequential computation의 제약 & 병렬성 이슈

[기존 모델의 한계 1_RNN 모델의 process]

- symbol의 위치를 계산 시간에 맞춰 정렬

  → 병렬성을 가로막는다

  → 긴 sequence lengths에 취약

  → 메모리 이슈

  ⇒ 성능 문제는 해결O, 근본적인 계산 효율 문제 해결X

[기존 모델의 한계 2_Attention mechanisms의 RNN에의 결합의 한계]

- 거리감에 대한 이해 없이 종속성 X

논문의 contributions

Transformer의 제안

- input과 output 사이의 종속성을 기반으로 attention에 의존
- 병렬화에 굿
- 번역 품질에 SOTA

2. Background

💡 Introduction에서 언급한 기존 연구들에 대해 어떻게 서술하는가
제안 방법의 차별성을 어떻게 표현하고 있는가

선행연구의 한계 in point of reducing sequential computation

[CNN 기반의 모델 Extended Neural GPU, ConvS2S, ByteNet]

- 은닉층에 대하여 병렬화는 가능..,,
- input과 output의 position distance가 커짐에 따라 연산수가 증가→ 거리 종속성 계산이 어려워짐

Operation에서의 Transformer의 차별성

- attention-weighted position의 평균 구하기 → 연산수 down

self-attention

- 서로 **다른** 위치의 각 sequence를 계산하기 위해 relate한 attention mechanism

End-to-end memory networks

- recurrent attention mechanism에 기반하여 간단한 QnA문제에 탁월한 네트워크

Transformer의 의의

- self-attention에 전적으로 의존한 첫 transduction model → 정렬된 sequence를 사용한 RNN과 convolution X
- 병렬화 O

3. 제안 방법론

💡 Introduction에서 언급된 내용과 동일하게 작성되어 있는가
Introduction에서 언급한 제안 방법이 가지는 장점에 대한 근거가 있는가
제안 방법에 대한 설명이 구현 가능하도록 작성되어 있는가

Main Idea

> RNN・CNN 기반 encoder-decoder구조에서 나아가 **Attention mechanism으로만** 구성된 새로운 **Transformer** 구조 제안

[multi-head Attention의 도입]

: 다른 위치의 다른 subspace에서 동시에 att을 수행하여 더 풍부한 문맥 정보를 추출 가능하도록

- encoder-decoder layer에의 도입
  - Q : 이전 decoder에서 / K・V : encoder의 결과
  - 모든 position의 decoder가 input sequence의 모든 위치에 개입 가능하도록

  → seq-2-seq 모델의 전형적인 encoder-decoder mechanism 미믹
- encoder에 self-att layer 포함

  → 모든 Q, K, V가 이전 layer output이라는 **same place**에서 옴.

  → encoder의 각각의 position이 encoder의 **모든 previous layer**에 개입 가능하도록
- decoder의 self-att
  - masking terminology 참고
  - masking 작업으로 이전의 info만 참고하도록

⇒ 장거리 의존성을 효율적으로 학습

[Positional Encoding]

❓Warum : T는 Recurrence와 Convolution을 포함하지 않아 seq token의 절대적・상대적 위치 정보가 필요.

→ Positional Encoding으로 **순서 정보의 활용**을 가능하게 함.

Contribution

Recurrence와 Convolution의 배제

: RNN・CNN 구조를 배제하고 **Attention mechanism으로만** 구성된 새로운 **Transformer** 구조 제안

성능 및 계산 효율성 개선

: Big model의 경우 기존 RNN, CNN 단일 모델 및 앙상블 모델보다 training cost 감소 + 성능 개선(높은 BLUE 점수 및 SOTA 달성) 달성

: Base model 또한 기존 RNN, CNN 단일 모델 및 앙상블 모델보다 성능이 개선됨

: 병렬화 달성

multi-head Attention의 도입

: 기존 attention의 한계 극복→ 장거리 의존성을 효율적으로 극복, 다양한 관점의 문맥 파악 가능

Positional Encoding의 도입

: Att에 위치 정보를 도입 → sequence 구조가 없는 한계를 극복

4. 실험 및 결과

💡 Introduction에서 언급한 제안 방법의 장점을 검증하기 위한 실험이 있는가

Dataset

[EN-DE translation]

- standard WMT 2014 EN-DE dataset : 4.5M 문장 페어, 공유된 source-target 단어는 37,000tokens

[EN-FE translation]

- Large WMT 2014 EN-FE dataset : 36M 문장 페어

[Batch]

- approx. 25k source tokens + 25k target tokens

Baseline

- RNN 및 CNN 기반의 base models 및 ensemble models etc.ByteNet, GNMT, ConvS2S
- 비교 점수 : BLUE 점수, Training cost with FLOPs

결과

Machine Translation에서의 결과 및 param

[EN-DE translation]

- Big model : BLUE 점수가 다른 single 모델 및 앙상블보다 2.0 높게 나옴(**SOTA**, 28.4점), 3.5일만에 학습
- Base model : BLUE 점수가 다른 모델 및 앙상블보다 높게 나옴. & cost도 low

[EN-FE translation]

- Big model : BLUE 점수가 다른 single 모델보다 outperforming함(**SOTA**, 41.0점) & cost도 1/4
    - Pdrop=0.1(0.3대신)

*사용된 param

- checkpoint : base=5 checkpoints, big=20 checkpoints
- beam search=4, length penalty=0.6
- hyper param → 실험 후 계속 조정
- max lenght → 추론하면서 조정, input보다 +50

English Constituency Parsing

- WSJ training set의 40K 문장으로 훈련했음에도 불구하고 모델 perform이 좋게 나온다 → RNN보다 성능 굿

5. 결론 (배운점)

💡 연구의 의의 및 한계점, 본인이 생각하는 좋았던/아쉬웠던 점 (배운점)

- RNN・CNN 구조를 배제하고 att. mechanism으로 모델 구현 및 번역 task 성능 개선
- 선행 연구의 병렬화와 거리감에 대한 이해 한계 극복
- 앞으로의 GPT, BERT 등 LLM의 기반이 된 중요한 논문